Informe técnico: consultas SQL y vistas de la red social estudiantil pascualina

Base de datos red_social · PostgreSQL / pgAdmin 4 · Script: consultas_y_vistas_red_social.sql

1. Objetivo y alcance

Este informe justifica las decisiones tomadas al construir el sistema de consultas DML y las vistas analíticas sobre la base de datos de la red social estudiantil pascualina (7 entidades y 6 tablas asociativas, 13 tablas en total). El objetivo es responder preguntas del modelo de negocio (actividad, popularidad, mentorías, grupos, eventos) y demostrar cómo las vistas permiten reutilizar consultas complejas en analítica de datos.

El script tiene cuatro bloques: (0) datos de prueba ampliados, (1) 18 consultas de negocio, (2) 9 vistas, (3) 11 consultas que reutilizan las vistas y (4) ejemplos de UPDATE/DELETE transaccionales.

2. Reglas de nomenclatura para ALIAS

Todo dato calculado lleva AS explícito y sigue estas reglas, para que cualquier persona deduzca el contenido y el tipo del campo con solo leer su nombre:

Regla	**Uso**	**Ejemplo**
snake_case en minúscula, sin tildes ni espacios	Todos los alias	nombre_completo
Prefijo total_	Conteos y sumas	total_seguidores
Prefijo promedio_	Promedios	promedio_semestre
Prefijo pct_	Porcentajes de 0 a 100	pct_asistencia
Sufijo _maximo / _minimo	MAX / MIN	semestre_maximo
Nombre descriptivo del rol	Cuando una tabla se usa dos veces	aprendiz, mentor
Prefijo vw_	Nombre de las vistas	vw_perfil_estudiante

Tipos de datos de los campos calculados

Alias	Expresión	Tipo resultante
nombre_completo	nombre || ' ' || apellido	TEXT
edad	EXTRACT(YEAR FROM age(...))::INTEGER	INTEGER
total_*	COUNT(*)	BIGINT
promedio_*, pct_*	ROUND(AVG(...), n)	NUMERIC
nivel_numerico	CASE nivel ... END	INTEGER
dias_de_mentoria	CURRENT_DATE - fecha_inicio	INTEGER
mes	TO_CHAR(DATE_TRUNC(...), 'YYYY-MM')	TEXT

3. Catálogo de consultas y justificación

ID	Pregunta de negocio	Técnica principal
CQ-01	¿Quién es la comunidad y qué edad tiene?	Concatenación, age()
CQ-02	¿Cómo se distribuye por carrera?	GROUP BY, AVG, MIN, MAX
CQ-03	¿Qué publicaciones generan más interacción?	CTE + LEFT JOIN + COALESCE
CQ-04	¿Quiénes son los más activos?	Subconsultas escalares en tabla derivada
CQ-05 / 06	¿Qué contenido y qué reacciones predominan?	COUNT + ventana SUM() OVER ()
CQ-07	¿Quiénes tienen más seguidores?	LEFT JOIN, COUNT ... FILTER
CQ-08	¿Qué mentorías existen?	Autojoin (estudiante dos veces vía sigue)
CQ-09 / 10	¿Qué habilidades abundan y quién puede ser mentor?	CASE, FILTER, subconsulta
CQ-11	¿Quiénes comparten intereses?	Autojoin con a.id < b.id, STRING_AGG, HAVING
CQ-12	¿Qué tan grandes son los grupos?	LEFT JOIN, conteo por rol
CQ-13 / 14	¿Qué eventos vienen y cómo fue la asistencia?	Filtro temporal, NULLIF, FILTER
CQ-15 / 16	¿Quién o qué no tiene actividad?	NOT EXISTS, anti-join LEFT JOIN ... IS NULL
CQ-17	¿Cómo evoluciona la publicación por mes?	DATE_TRUNC, TO_CHAR
CQ-18	¿A quién recomendar seguir?	Autojoin de participa_en + NOT EXISTS

Las consultas cubren los operadores relacionales clave: proyección, selección, INNER/LEFT JOIN, autojoin, agregación con GROUP BY/HAVING, subconsultas correlacionadas, CTE y funciones de ventana.

4. Vistas y justificación

Vista	Qué resuelve	Por qué una vista
vw_perfil_estudiante	Perfil 360° con 8 métricas por estudiante	Es la base de otras vistas; evita repetir 8 subconsultas
vw_engagement_publicacion	Comentarios, reacciones e interacción total	Es la métrica central de una red social
vw_red_mentoria	Pares aprendiz–mentor	Oculta el autojoin y el filtro es_mentoria
vw_habilidades_estudiante	Habilidades con nivel numérico	Permite promediar un dominio que se guarda como texto
vw_afinidad_intereses	Pares con intereses comunes	Base de un sistema de recomendación
vw_resumen_grupo	Tamaño, roles y eventos del grupo	Combina dos agregaciones independientes
vw_eventos_asistencia	Estado del evento y tasa de asistencia	Calcula el estado según la hora actual
vw_resumen_carrera	Indicadores por carrera	Se construye sobre otra vista (composición)
vw_estudiantes_inactivos	Estudiantes sin actividad	Lista lista para campañas de reactivación

Las vistas no almacenan datos: siempre reflejan el estado actual de las tablas, y vw_eventos_asistencia recalcula el estado del evento con now() en cada consulta. Además simplifican el trabajo de quien analiza datos (basta un SELECT sobre la vista) y centralizan la lógica, de modo que si una regla cambia, se modifica en un solo lugar.

5. Decisiones técnicas relevantes

1. **Evitar la multiplicación de filas.** Unir a la vez comentarios y reacciones de una publicación produciría un producto cartesiano y conteos inflados. Se resolvió agregando cada tabla por separado (CTE o subconsulta derivada) antes de unirla.
2. **LEFT JOIN + COALESCE.** Las publicaciones sin comentarios, los grupos sin miembros y los estudiantes sin seguidores deben aparecer con 0 y no desaparecer del resultado.

3. **COUNT(*) FILTER (WHERE ...).** Es la forma estándar de PostgreSQL para conteos condicionales; es más legible que SUM(CASE ...) y evita varias consultas.
4. **NULLIF en los porcentajes.** Evita la división por cero cuando un evento no tiene inscritos.
5. **Nivel como número.** tiene_habilidad.nivel se guarda como texto con CHECK (decisión del diseño físico); para promediarlo se mapea a 1, 2 y 3 en la consulta y no se cambia el modelo.
6. **Autojoin con a.id < b.id.** Evita que cada pareja aparezca dos veces (A–B y B–A) y que alguien se empareje consigo mismo.
7. **NOT EXISTS frente a NOT IN.** Es seguro ante valores NULL y suele ser más eficiente.
8. **Scripts re-ejecutables.** Cada vista se precede de DROP VIEW IF EXISTS en orden inverso de dependencia.
9. **Aprovechamiento de los índices.** Los filtros y las uniones usan columnas ya indexadas (ix_sigue_seguido, ix_reacciona_publicacion, ix_asiste_evento, ix_evento_inicio), por lo que las consultas siguen escalando cuando crezcan los datos.
10. **DML en transacción.** Los UPDATE/DELETE de ejemplo se ejecutan entre BEGIN y ROLLBACK: demuestran la sintaxis y el efecto sobre las vistas sin alterar los datos de prueba.

6. Validación

- El script se ejecuta completo sobre la base recién creada con el DDL; se verificó que cada consulta devuelve filas con los datos de prueba ampliados (6 estudiantes nuevos, 6 publicaciones, 10 comentarios numerados por el trigger, 16 reacciones, 11 membresías, etc.).
- Los datos incluyen casos límite a propósito: un estudiante sin actividad (Juan Díaz), una publicación sin comentarios, un evento independiente (id_grupo nulo), un evento ya realizado y una inscripción cancelada.
- Los porcentajes y totales se pueden cotejar a mano con los INSERT del bloque 0.

7. Conclusión

El sistema de consultas traduce las reglas del modelo (participaciones, cardinalidades, relaciones recursivas y entidad débil) en preguntas concretas de negocio. Las vistas encapsulan la lógica más compleja y la hacen reutilizable, mientras que la nomenclatura uniforme de los alias hace que los resultados sean auto explicativos. El conjunto queda listo para alimentar tableros o informes y para crecer con nuevas vistas (por ejemplo, vistas materializadas si el volumen lo exige).